Search models, datasets, users...

Deploying on AWS documentation
Deploy models to Amazon SageMaker

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

Deploy models to Amazon SageMaker

Deploying a 🤗 Transformers models in SageMaker for inference is as easy as:

```
from sagemaker.huggingface import HuggingFaceModel

# create Hugging Face Model Class and deploy it as SageMaker endpoint
huggingface_model = HuggingFaceModel(...).deploy()
```

This guide will show you how to deploy models with zero-code using the Inference Toolkit. The Inference Toolkit builds on top of the `pipeline` feature from 🤗 Transformers. Learn how to:

- Install and setup the Inference Toolkit.
- Deploy a 🤗 Transformers model trained in SageMaker.
- Deploy a 🤗 Transformers model from the Hugging Face [model Hub](https://huggingface.co/models).
- Run a Batch Transform Job using 🤗 Transformers and Amazon SageMaker.
- Create a custom inference module.

Installation and setup

Before deploying a 🤗 Transformers model to SageMaker, you need to sign up for an AWS account. If you don't have an AWS account yet, learn more here.

Once you have an AWS account, get started using one of the following:

- SageMaker Studio
- SageMaker notebook instance
- Local environment

To start training locally, you need to setup an appropriate IAM role.

Upgrade to the latest `sagemaker` version.

```
pip install sagemaker --upgrade
```

SageMaker environment

Setup your SageMaker environment as shown below:

```
import sagemaker
sess = sagemaker.Session()
role = sagemaker.get_execution_role()
```

Note: The execution role is only available when running a notebook within SageMaker. If you run `get_execution_role` in a notebook not on SageMaker, expect a `region` error.

Local environment

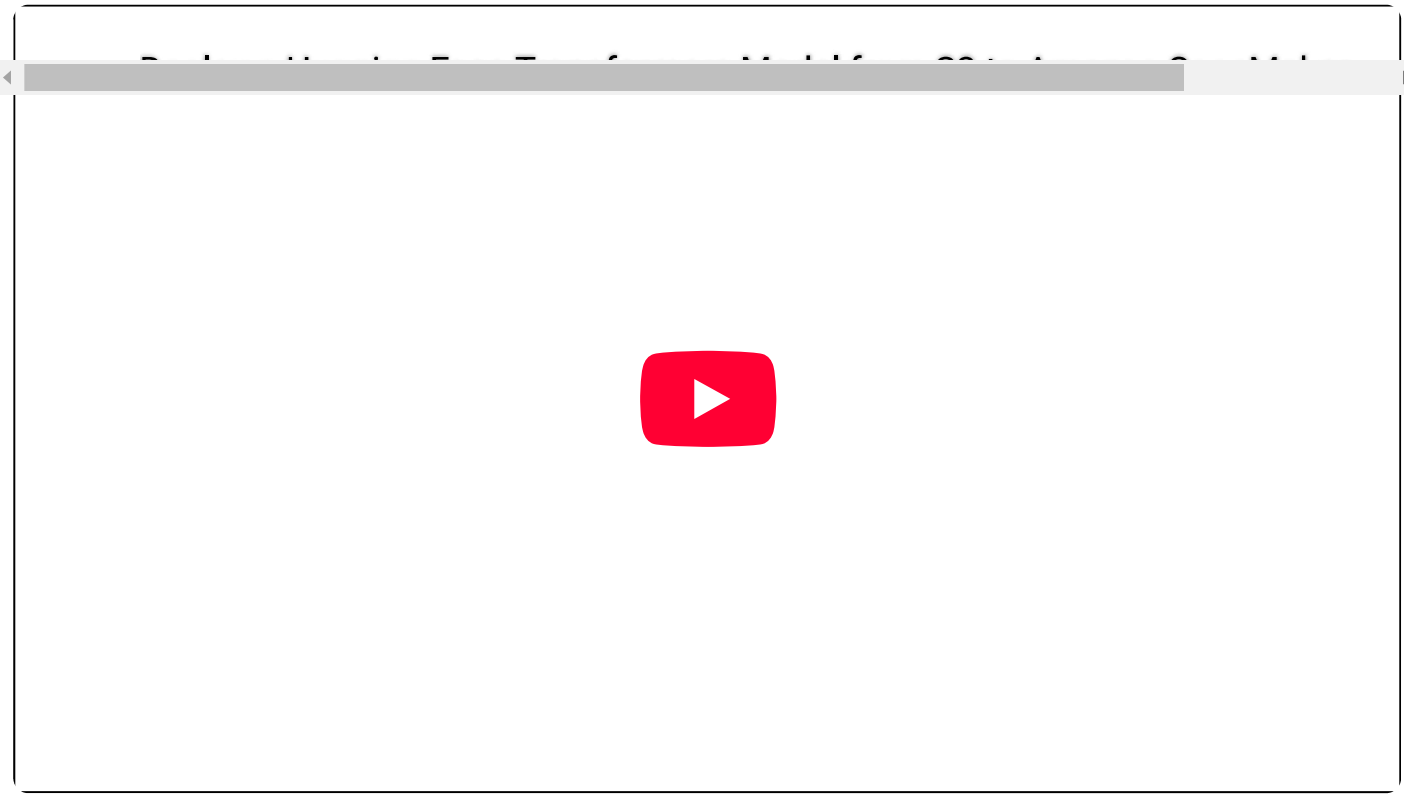
Setup your local environment as shown below:

```
import sagemaker
import boto3

iam_client = boto3.client('iam')
```

```
role = iam_client.get_role(RoleName='role-name-of-your-iam-role-with-right-permissions')[
sess = sagemaker.Session()
```

Deploy a 🤗 Transformers model trained in SageMaker



There are two ways to deploy your Hugging Face model trained in SageMaker:

- Deploy it after your training has finished.
- Deploy your saved model at a later time from S3 with the `model_data`.

📖 Open the deploy_transformer_model_from_s3.ipynb notebook for an example of how to deploy a model from S3 to SageMaker for inference.

Deploy after training

To deploy your model directly after training, ensure all required files are saved in your training script, including the tokenizer and the model.

If you use the Hugging Face `Trainer`, you can pass your tokenizer as an argument to the `Trainer`. It will be automatically saved when you call `trainer.save_model()`.

```
from sagemaker.huggingface import HuggingFace

############ pseudo code start ############

# create Hugging Face Estimator for training
huggingface_estimator = HuggingFace(....)

# start the train job with our uploaded datasets as input
huggingface_estimator.fit(...)

############ pseudo code end ############

# deploy model to SageMaker Inference
predictor = hf_estimator.deploy(initial_instance_count=1, instance_type="ml.m5.xlarge")

# example request: you always need to define "inputs"
data = {
   "inputs": "Camera - You are awarded a SiPix Digital Camera! call 09061221066 fromm lar
}

# request
predictor.predict(data)
```

After you run your request you can delete the endpoint as shown:

```
# delete endpoint
predictor.delete_endpoint()
```

Deploy with model_data

If you’ve already trained your model and want to deploy it at a later time, use the `model_data` argument to specify the location of your tokenizer and model weights.

```
from sagemaker.huggingface.model import HuggingFaceModel
```

```
# create Hugging Face Model Class
huggingface_model = HuggingFaceModel(
   model_data="s3://models/my-bert-model/model.tar.gz",  # path to your trained SageMaker
   role=role,                                            # IAM role with permissions to c
   transformers_version="4.26",                           # Transformers version used
   pytorch_version="1.13",                                # PyTorch version used
   py_version='py39',                                    # Python version used
)

# deploy model to SageMaker Inference
predictor = huggingface_model.deploy(
   initial_instance_count=1,
   instance_type="ml.m5.xlarge"
)

# example request: you always need to define "inputs"
data = {
   "inputs": "Camera - You are awarded a SiPix Digital Camera! call 09061221066 fromm lar
}

# request
predictor.predict(data)
```

After you run our request, you can delete the endpoint again with:

```
# delete endpoint
predictor.delete_endpoint()
```

Create a model artifact for deployment

For later deployment, you can create a `model.tar.gz` file that contains all the required files, such as:

- `pytorch_model.bin`
- `tf_model.h5`
- `tokenizer.json`
- `tokenizer_config.json`

For example, your file should look like this:

```
model.tar.gz/
|- pytorch_model.bin
|- vocab.txt
|- tokenizer_config.json
|- config.json
|- special_tokens_map.json
```

Create your own `model.tar.gz` from a model from the 🤗 Hub:

1. Download a model:

```
git lfs install
git clone git@hf.co:{repository}
```

2. Create a `tar` file:

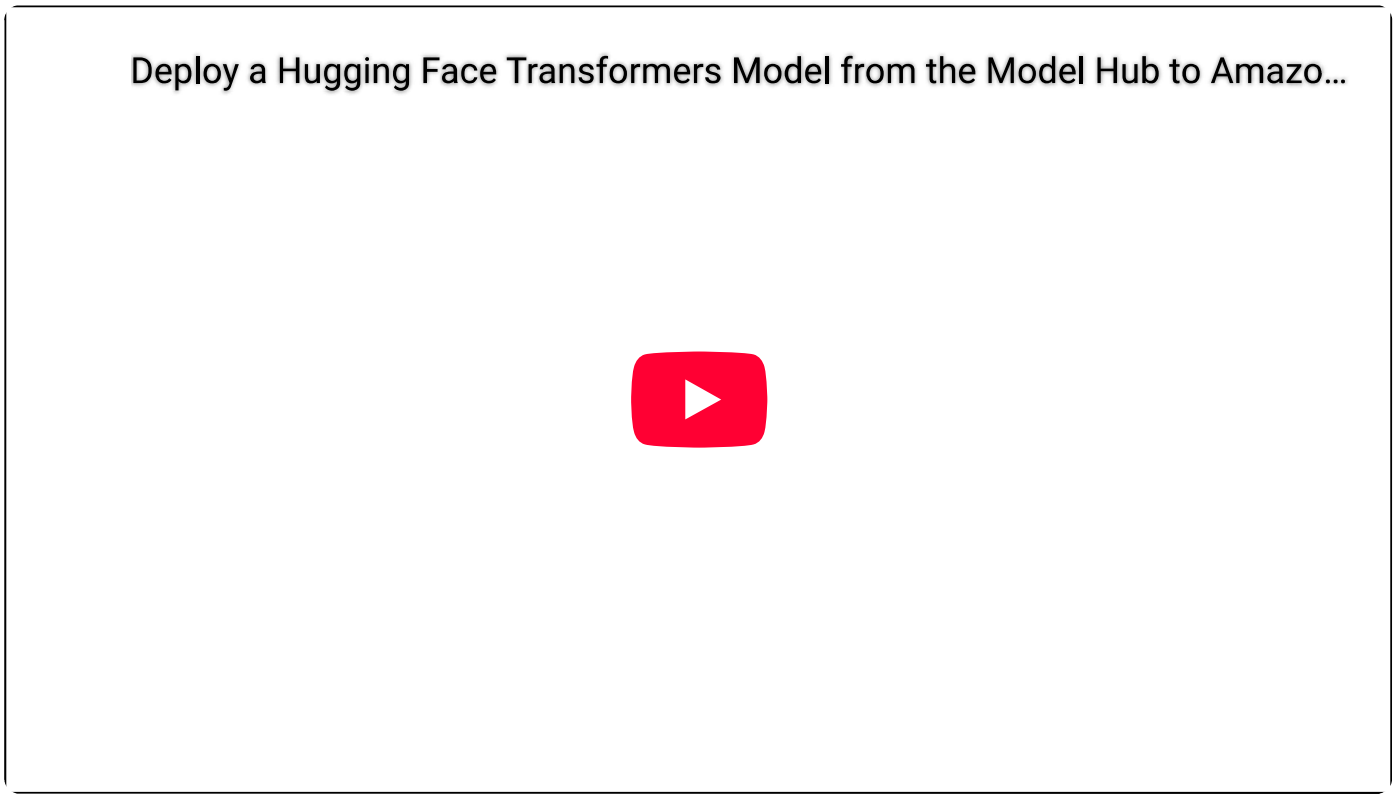
```
cd {repository}
tar zcvf model.tar.gz *
```

3. Upload `model.tar.gz` to S3:

```
aws s3 cp model.tar.gz <s3://{my-s3-path}>
```

Now you can provide the S3 URI to the `model_data` argument to deploy your model later.

Deploy a model from the 🤗 Hub



To deploy a model directly from the 🤗 Hub to SageMaker, define two environment variables when you create a `HuggingFaceModel`:

- `HF_MODEL_ID` defines the model ID which is automatically loaded from huggingface.co/models when you create a SageMaker endpoint. Access 10,000+ models on he 🤗 Hub through this environment variable.
- `HF_TASK` defines the task for the 🤗 Transformers `pipeline`. A complete list of tasks can be found here.

> *" ⚠️ ** Pipelines are not optimized for parallelism (multi-threading) and tend to consume a lot of RAM. For example, on a GPU-based instance, the pipeline operates on a single vCPU. When this vCPU becomes saturated with the inference requests preprocessing, it can create a bottleneck, preventing the GPU from being fully utilized for model inference. Learn more here"*

```
from sagemaker.huggingface.model import HuggingFaceModel

# Hub model configuration <https://huggingface.co/models>
hub = {
  'HF_MODEL_ID':'distilbert-base-uncased-distilled-squad', # model_id from hf.co/models
```

```
  'HF_TASK':'question-answering'                          # NLP task you want to use for
}

# create Hugging Face Model Class
huggingface_model = HuggingFaceModel(
   env=hub,                                               # configuration for loading mo
   role=role,                                             # IAM role with permissions to
   transformers_version="4.26",                            # Transformers version used
   pytorch_version="1.13",                                 # PyTorch version used
   py_version='py39',                                     # Python version used
)

# deploy model to SageMaker Inference
predictor = huggingface_model.deploy(
   initial_instance_count=1,
   instance_type="ml.m5.xlarge"
)

# example request: you always need to define "inputs"
data = {
"inputs": {
    "question": "What is used for inference?",
    "context": "My Name is Philipp and I live in Nuremberg. This model is used with sagem
    }
}

# request
predictor.predict(data)
```

After you run our request, you can delete the endpoint again with:

```
# delete endpoint
predictor.delete_endpoint()
```

📓 Open the deploy_transformer_model_from_hf_hub.ipynb notebook for an example of how to deploy a model from the 🤗 Hub to SageMaker for inference.

Run batch transform with 🤗 Transformers and SageMaker

Run a Batch Transform Job using Hugging Face Transformers and Amazon ...

After training a model, you can use SageMaker batch transform to perform inference with the model. Batch transform accepts your inference data as an S3 URI and then SageMaker will take care of downloading the data, running the prediction, and uploading the results to S3. For more details about batch transform, take a look here.

⚠️ The Hugging Face Inference DLC currently only supports `.jsonl` for batch transform due to the complex structure of textual data.

Note: Make sure your `inputs` fit the `max_length` of the model during preprocessing.

If you trained a model using the Hugging Face Estimator, call the `transformer()` method to create a transform job for a model based on the training job (see here for more details):

```
batch_job = huggingface_estimator.transformer(
    instance_count=1,
    instance_type='ml.p3.2xlarge',
    strategy='SingleRecord')


batch_job.transform(
    data='s3://s3-uri-to-batch-data',
```

```
    content_type='application/json',
    split_type='Line')
```

If you want to run your batch transform job later or with a model from the 🤗 Hub, create a `HuggingFaceModel` instance and then call the `transformer()` method:

```
from sagemaker.huggingface.model import HuggingFaceModel

# Hub model configuration <https://huggingface.co/models>
hub = {
    'HF_MODEL_ID':'distilbert/distilbert-base-uncased-finetuned-sst-2-english',
    'HF_TASK':'text-classification'
}

# create Hugging Face Model Class
huggingface_model = HuggingFaceModel(
   env=hub,                                                # configuration for loading mo
   role=role,                                              # IAM role with permissions to
   transformers_version="4.26",                             # Transformers version used
   pytorch_version="1.13",                                  # PyTorch version used
   py_version='py39',                                      # Python version used
)

# create transformer to run a batch job
batch_job = huggingface_model.transformer(
    instance_count=1,
    instance_type='ml.p3.2xlarge',
    strategy='SingleRecord'
)

# starts batch transform job and uses S3 data as input
batch_job.transform(
    data='s3://sagemaker-s3-demo-test/samples/input.jsonl',
    content_type='application/json',
    split_type='Line'
)
```

The `input.jsonl` looks like this:

```
{"inputs":"this movie is terrible"}
{"inputs":"this movie is amazing"}
{"inputs":"SageMaker is pretty cool"}
{"inputs":"SageMaker is pretty cool"}
{"inputs":"this movie is terrible"}
{"inputs":"this movie is amazing"}
```

📓 Open the sagemaker-notebook.ipynb notebook for an example of how to run a batch transform job for inference.

Deploy an LLM to SageMaker using TGI

If you are interested in using a high-performance serving container for LLMs, you can use the Hugging Face TGI container. This utilizes the Text Generation Inference library. A list of compatible models can be found here.

First, make sure that the latest version of SageMaker SDK is installed:

```
pip install sagemaker>=2.231.0
```

Then, we import the SageMaker Python SDK and instantiate a sagemaker_session to find the current region and execution role.

```
import sagemaker
from sagemaker.huggingface import HuggingFaceModel, get_huggingface_llm_image_uri
import time

sagemaker_session = sagemaker.Session()
region = sagemaker_session.boto_region_name
role = sagemaker.get_execution_role()
```

Next we retrieve the LLM image URI. We use the helper function get_huggingface_llm_image_uri() to generate the appropriate image URI for the Hugging Face Large Language Model (LLM) inference. The function takes a required parameter backend and several optional parameters. The backend

specifies the type of backend to use for the model: “huggingface” refers to using Hugging Face TGI backend.

```
image_uri = get_huggingface_llm_image_uri(
  backend="huggingface",
  region=region
)
```

Now that we have the image uri, the next step is to configure the model object. We specify a unique name, the image_uri for the managed TGI container, and the execution role for the endpoint. Additionally, we specify a number of environment variables including the `HF_MODEL_ID` which corresponds to the model from the HuggingFace Hub that will be deployed, and the `HF_TASK` which configures the inference task to be performed by the model.

You should also define `SM_NUM_GPUS`, which specifies the tensor parallelism degree of the model. Tensor parallelism can be used to split the model across multiple GPUs, which is necessary when working with LLMs that are too big for a single GPU. To learn more about tensor parallelism with inference, see our previous blog post. Here, you should set `SM_NUM_GPUS` to the number of available GPUs on your selected instance type. For example, in this tutorial, we set `SM_NUM_GPUS` to 4 because our selected instance type ml.g4dn.12xlarge has 4 available GPUs.

Note that you can optionally reduce the memory and computational footprint of the model by setting the `HF_MODEL_QUANTIZE` environment variable to `true`, but this lower weight precision could affect the quality of the output for some models.

```
model_name = "llama-3-1-8b-instruct" + time.strftime("%Y-%m-%d-%H-%M-%S", time.gmtime())

hub = {
    'HF_MODEL_ID':'meta-llama/Llama-3.1-8B-Instruct',
    'SM_NUM_GPUS':'1',
    'HUGGING_FACE_HUB_TOKEN': '<REPLACE WITH YOUR TOKEN>',
}

assert hub['HUGGING_FACE_HUB_TOKEN'] != '<REPLACE WITH YOUR TOKEN>', "You have to provide


model = HuggingFaceModel(
```

```
    name=model_name,
    env=hub,
    role=role,
    image_uri=image_uri
)
```

Next, we invoke the deploy method to deploy the model.

```
predictor = model.deploy(
  initial_instance_count=1,
  instance_type="ml.g5.2xlarge",
  endpoint_name=model_name
)
```

Once the model is deployed, we can invoke it to generate text. We pass an input prompt and run the

```
input_data = {
  "inputs": "The diamondback terrapin was the first reptile to",
  "parameters": {
    "do_sample": True,
    "max_new_tokens": 100,
    "temperature": 0.7,
    "watermark": True
  }
}

predictor.predict(input_data)
```

We receive the following auto-generated text response:

```
[{'generated_text': 'The diamondback terrapin was the first reptile to make the list, fol
```

Once we are done experimenting, we delete the endpoint and the model resources.

```
predictor.delete_model()
predictor.delete_endpoint()
```

User defined code and modules

The Hugging Face Inference Toolkit allows the user to override the default methods of the `HuggingFaceHandlerService`. You will need to create a folder named `code/` with an `inference.py` file in it. See here for more details on how to archive your model artifacts. For example:

```
model.tar.gz/
|- pytorch_model.bin
|- ....
|- code/
  |- inference.py
  |- requirements.txt
```

The `inference.py` file contains your custom inference module, and the `requirements.txt` file contains additional dependencies that should be added. The custom module can override the following methods:

- `model_fn(model_dir)` overrides the default method for loading a model. The return value `model` will be used in `predict` for predictions. `predict` receives argument the `model_dir`, the path to your unzipped `model.tar.gz`.
- `transform_fn(model, data, content_type, accept_type)` overrides the default transform function with your custom implementation. You will need to implement your own `preprocess`, `predict` and `postprocess` steps in the `transform_fn`. This method can't be combined with `input_fn`, `predict_fn` or `output_fn` mentioned below.
- `input_fn(input_data, content_type)` overrides the default method for preprocessing. The return value `data` will be used in `predict` for predictions. The inputs are:
  - `input_data` is the raw body of your request.
  - `content_type` is the content type from the request header.

- `predict_fn(processed_data, model)` overrides the default method for predictions. The return value `predictions` will be used in `postprocess`. The input is `processed_data`, the result from `preprocess`.
- `output_fn(prediction, accept)` overrides the default method for postprocessing. The return value `result` will be the response of your request (e.g. `JSON`). The inputs are:
  - `predictions` is the result from `predict`.
  - `accept` is the return accept type from the HTTP Request, e.g. `application/json`.

Here is an example of a custom inference module with `model_fn`, `input_fn`, `predict_fn`, and `output_fn`:

```
from sagemaker_huggingface_inference_toolkit import decoder_encoder

def model_fn(model_dir):
    # implement custom code to load the model
    loaded_model = ...

    return loaded_model

def input_fn(input_data, content_type):
    # decode the input data  (e.g. JSON string -> dict)
    data = decoder_encoder.decode(input_data, content_type)
    return data

def predict_fn(data, model):
    # call your custom model with the data
    outputs = model(data , ... )
    return predictions

def output_fn(prediction, accept):
    # convert the model output to the desired output format (e.g. dict -> JSON string)
    response = decoder_encoder.encode(prediction, accept)
    return response
```

Customize your inference module with only `model_fn` and `transform_fn`:

```python
from sagemaker_huggingface_inference_toolkit import decoder_encoder

def model_fn(model_dir):
    # implement custom code to load the model
    loaded_model = ...

    return loaded_model

def transform_fn(model, input_data, content_type, accept):
     # decode the input data (e.g. JSON string -> dict)
    data = decoder_encoder.decode(input_data, content_type)

    # call your custom model with the data
    outputs = model(data , ... )

    # convert the model output to the desired output format (e.g. dict -> JSON string)
    response = decoder_encoder.encode(output, accept)

    return response
```

<> Update on GitHub

Hugging Face on AWS →